COAHSY

Documentación Técnica

Generado: 13/10/2026 05:10 UTC

AutoDocs AI

Tabla de Contenido

Tabla de Contenido 2

Documentación del Proyecto 5

Resumen Ejecutivo . . . 5

Stack Tecnológico . . . 5

Estadísticas del Proyecto . . . 5

2. Documentación de Arquitectura y Diseño . . . 6

2.1 Estructura de Directorios . . . 6

2.2 Distribución por Capas . . . 6

2.3 Distribución de Lenguajes . . . 6

2.4 Diagrama de Infraestructura . . . 7

2.5 Diagrama de Casos de Uso . . . 7

2.6 Diagrama de Clases . . . 8

2.7 Patrones Detectados . . . 8

2.8 Insights de Arquitectura (IA) . . . 8

3. Documentación Técnica de Código . . . 8

3.1 Guía de Configuración del Entorno de Desarrollo . . . 8

3.2 Documentación de API . . . 9

3.3 Estado de Comentarios y Docstrings . . . 18

3.4 Planes de Prueba (Testing) . . . 18

4. Documentación de Clases . . . 19

`Config` . . . 20

`Usuario` . . . 20

`Grupo` *20*

`Deportista` *20*

`Dispositivo` *21*

`RegistroActividad` *21*

`MetaRutina` *21*

`Notificacion` *21*

5. Documentación de Funciones 22

Módulo: `app.py` *22*

Módulo: `utils.py` *23*

Módulo: `auth_utils.py` *24*

Módulo: `routes/historial.py` *25*

Módulo: `routes/auth.py` *26*

Módulo: `routes/actividad.py` *26*

Módulo: `routes/notificaciones.py` *28*

Módulo: `routes/grupos.py` *29*

Módulo: `routes/metas.py` *29*

Módulo: `routes/dispositivos.py` *30*

Módulo: `routes/deportistas.py` *31*

Módulo: `web/js/app.js` *32*

6. Modelos de Datos 36

Esquema General Inferido *36*

7. Guía de Despliegue y Mantenimiento 36

7.1 Requisitos del Sistema *36*

7.2 Instalación Local *36*

7.3 Despliegue con Docker *37*

7.4 Variables de Entorno Recomendadas 37

7.5 Checklist de Despliegue 37

7.6 Registro de Cambios (Changelog) 38

8. Reporte de Calidad del Código 38

8.1 Análisis de Métricas 38

8.2 Funciones con Alta Complejidad Ciclomática 39

8.3 Recomendaciones 39

8.4 Errores de Análisis 39

9. Análisis de Datos del Proyecto 39

9.1 Tipos y Fuentes de Datos Detectados 39

9.2 Esquema de Data Warehouse Inferido 39

9.3 Reporte de Calidad de Datos 39

9.4 Minería de Datos Aplicada al Código 39

9.5 Pipeline ETL Utilizado 43

Coahsy

Documentación Técnica · AutoDocs AI · 13/10/2026

20	63	8	20
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 75/100

Documentación del Proyecto

Generado automáticamente por AutoDocs AI — 2026-10-13 05:10 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con 20 archivos fuente, 63 funciones, 8 clases y 20 endpoints de API documentados. El score de calidad del código es **75/100**.

Stack Tecnológico

Lenguaje	Cantidad
Python	16 archivos
JSON	1 archivos
HTML	1 archivos
CSS	1 archivos
JavaScript	1 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	20
Funciones	63
Clases	8

Endpoints API	20
Score de calidad	75/100
Funciones documentadas	12 / 63
Clases documentadas	0 / 8

---

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/
■■■ (raíz)/
■■■ .expo/
■■■ routes/
■■■ web/
```

2.2 Distribución por Capas

• `(raíz)/` — 9 archivos [.md(1), .txt(1), .py(7)]

• `.expo/` — 2 archivos [.json(1), .md(1)]

• `routes/` — 9 archivos [.py(9)]

• `web/` — 5 archivos [.md(1), .svg(1), .html(1), .css(1), .js(1)]

2.3 Distribución de Lenguajes

• Python: 16 archivos (80%)

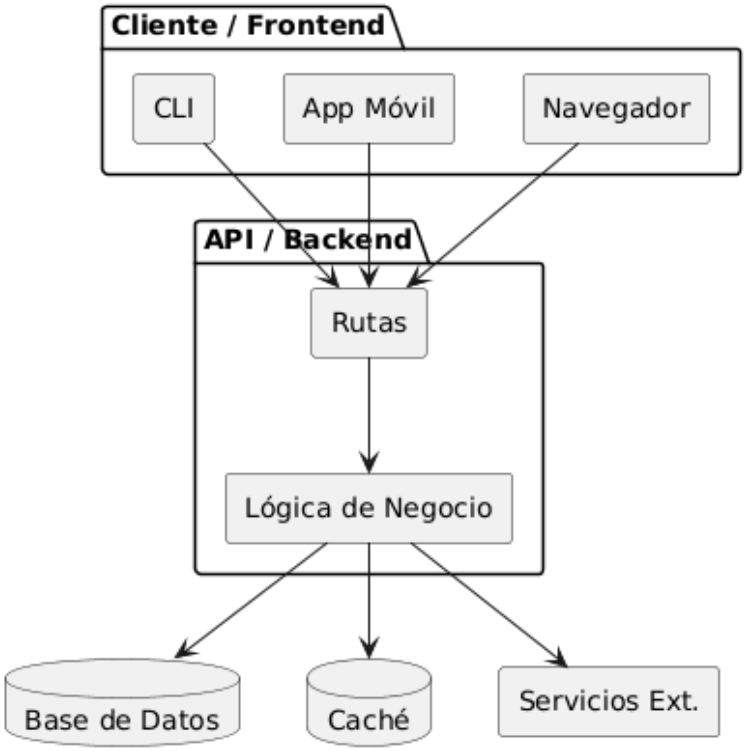
• JSON: 1 archivos (5%)

• HTML: 1 archivos (5%)

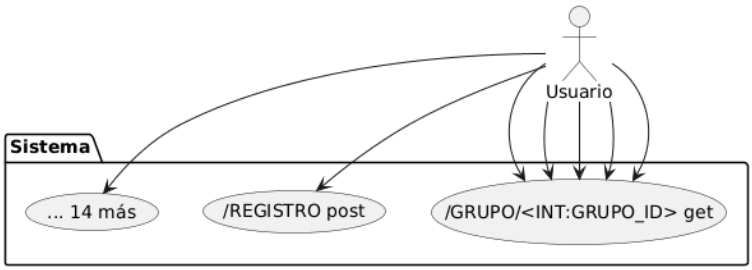
• CSS: 1 archivos (5%)

• JavaScript: 1 archivos (5%)

2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso



2.6 Diagrama de Clases



■■ Clases con información incompleta: Las siguientes clases aparecen sin métodos ni atributos documentados: `Config`, `Usuario`, `Grupo`, `Deportista`, `Dispositivo`, `RegistroActividad`, `MetaRutina`, `Notificacion`. Para que el diagrama se visualice correctamente, agrega métodos, propiedades o docstrings a estas clases en tu código fuente y regenera la documentación.

2.7 Patrones Detectados

• **API REST**: 20 endpoints detectados (Flask/FastAPI)

• **Programación Asíncrona**: 10 funciones async/await

2.8 Insights de Arquitectura (IA)

No se pudieron generar insights de arquitectura

---

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 3. Instalar dependencias
```

```
pip install -r requirements.txt

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py  # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: **20**

Módulo: `app.py`

[GET] `/ get`

• **Framework**: Flask/FastAPI

• **Archivo**: `app.py`

Endpoint / get

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/INDEX.HTML get`

• **Framework**: Flask/FastAPI

• **Archivo**: `app.py`

Endpoint /INDEX.HTML get

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa
• `400` — Solicitud incorrecta
• `401` — No autorizado
• `404` — Recurso no encontrado
• `500` — Error interno del servidor

[GET] `/FAVICON.ICO get`

• **Framework**: Flask/FastAPI
• **Archivo**: `app.py`

Endpoint /FAVICON.ICO get

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa
• `400` — Solicitud incorrecta
• `401` — No autorizado
• `404` — Recurso no encontrado
• `500` — Error interno del servidor

[GET] `/API/HEALTH get`

• **Framework**: Flask/FastAPI
• **Archivo**: `app.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa
• `400` — Solicitud incorrecta
• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/historial.py`

[GET] `/GRUPO/<INT:GRUPO_ID> get`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/historial.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/auth.py`

[GET] `/REGISTRO post`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/auth.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/LOGIN post`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/auth.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/actividad.py`

[GET] `/DEPORTISTA/<INT:DEPORTISTA_ID> get`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/actividad.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/GRUPO/<INT:GRUPO_ID> get`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/actividad.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/notificaciones.py`

[GET] `/DEPORTISTA/<INT:DEPORTISTA_ID> get`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/notificaciones.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/<INT:NOTIF_ID>/LEIDA patch`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/notificaciones.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/grupos.py`

[GET] `/<INT:GRUPO_ID> put`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/grupos.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/<INT:GRUPO_ID> delete`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/grupos.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/metas.py`

[GET] `/DEPORTISTA/<INT:DEPORTISTA_ID> get`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/metas.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/<INT:META_ID>/ESTADO patch`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/metas.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/dispositivos.py`

[GET] `/<INT:DISPOSITIVO_ID>/ESTADO patch`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/dispositivos.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/<INT:DISPOSITIVO_ID> delete`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/dispositivos.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

Módulo: `routes/deportistas.py`

[GET] `/<INT:DEPORTISTA_ID> get`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/deportistas.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/<INT:DEPORTISTA_ID> put`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/deportistas.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

[GET] `/<INT:DEPORTISTA_ID> delete`

• **Framework**: Flask/FastAPI

• **Archivo**: `routes/deportistas.py`

Parámetros esperados:

• `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

• `200` — Operación exitosa

• `400` — Solicitud incorrecta

• `401` — No autorizado

• `404` — Recurso no encontrado

• `500` — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 63

- **[OK] Documentadas:** 12 (19%)
- **[X] Sin documentar:** 51 (81%)

Visualización de documentación:

```
[■■■■■■■■■■■■■■■■■■■■]
19% documentado
```

Ejemplo de docstrings detectados:

```
def index():
    """
    Panel del entrenador (web/index.html).
    """
    ...
```

```
def serialize(obj):
    """
    Convierte un modelo SQLAlchemy (o lista de modelos) a dict JSON
-safe.
    """
    ...
```

```
def require_fields(data, fields):
    """
    Regresa lista de campos faltantes o vacíos.
    """
    ...
```

Funciones que requieren documentación:

- `create_app()` en `app.py`
- `favicon()` en `app.py`
- `health()` en `app.py`
- `not_found(e)` en `app.py`
- `method_not_allowed(e)` en `app.py`
- ... y 46 más

3.4 Planes de Prueba (Testing)

No se detectaron pruebas automatizadas. Se recomienda implementar:

```
# Ejemplo de estructura de tests recomendada
import pytest

class TestEndpoints:
    def test_get_returns_200(self, client):
        response = client.get('/api/endpoint')
        assert response.status_code == 200

    def test_post_creates_resource(self, client):
        data = {'name': 'test'}
        response = client.post('/api/endpoint', json=data)
        assert response.status_code == 201

    def test_unauthorized_returns_401(self, client):
        response = client.get('/api/protected')
        assert response.status_code == 401
```

Escenarios de prueba recomendados:

| Escenario | Tipo | Prioridad |

| --- | --- | --- |

| / get — respuesta 200 | Integración | Alta |

| / get — sin auth (401) | Seguridad | Alta |

| /INDEX.HTML get — respuesta 200 | Integración | Alta |

| /INDEX.HTML get — sin auth (401) | Seguridad | Alta |

| /FAVICON.ICO get — respuesta 200 | Integración | Alta |

| /FAVICON.ICO get — sin auth (401) | Seguridad | Alta |

| /API/HEALTH get — respuesta 200 | Integración | Alta |

| /API/HEALTH get — sin auth (401) | Seguridad | Alta |

| /GRUPO/<INT:GRUPO_ID> get — respuesta 200 | Integración | Alta |

| /GRUPO/<INT:GRUPO_ID> get — sin auth (401) | Seguridad | Alta |

| /REGISTRO post — respuesta 200 | Integración | Alta |

| /REGISTRO post — sin auth (401) | Seguridad | Alta |

---

4. Documentación de Clases

Total: **8 clases**

`Config`

- **Archivo:** `config.py`
- **Línea:** 3
- **Hereda de:** object
- **Métodos:** 0
- **Documentada:** [X] No

Clase Config

`Usuario`

- **Archivo:** `models.py`
- **Línea:** 6
- **Hereda de:** db.Model
- **Métodos:** 0
- **Documentada:** [X] No

Clase Usuario

`Grupo`

- **Archivo:** `models.py`
- **Línea:** 20
- **Hereda de:** db.Model
- **Métodos:** 0
- **Documentada:** [X] No

Clase Grupo

`Deportista`

- **Archivo:** `models.py`
- **Línea:** 30
- **Hereda de:** db.Model

• **Métodos:** 0

• **Documentada:** [X] No

`Dispositivo`

• **Archivo:** `models.py`

• **Línea:** 46

• **Hereda de:** db.Model

• **Métodos:** 0

• **Documentada:** [X] No

`RegistroActividad`

• **Archivo:** `models.py`

• **Línea:** 58

• **Hereda de:** db.Model

• **Métodos:** 0

• **Documentada:** [X] No

`MetaRutina`

• **Archivo:** `models.py`

• **Línea:** 71

• **Hereda de:** db.Model

• **Métodos:** 0

• **Documentada:** [X] No

`Notificacion`

• **Archivo:** `models.py`

• **Línea:** 84

• **Hereda de:** db.Model

• **Métodos:** 0

• **Documentada:** [X] No

5. Documentación de Funciones

Total: **63 funciones**

Módulo: `app.py`

`create_app`

• Parámetros: –

• Async: —

• Complejidad: [LOW] 2

• Documentada: [X]

Función create_app

`index`

• Parámetros: –

• Async: —

• Complejidad: [LOW] 1

• Documentada: [OK]

Panel del entrenador (web/index.html).

`favicon`

• Parámetros: –

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Función favicon

`health`

• Parámetros: –

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

Función health

`not_found`

- Parámetros: `e`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

Función not_found

`method_not_allowed`

- Parámetros: `e`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`server_error`

- Parámetros: `e`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`index()`

Ubicación: `app.py`

Panel del entrenador (web/index.html).

Módulo: `utils.py`

`serialize`

- Parámetros: `obj`
- Async: —
- Complejidad: [MED] 6
- Documentada: [OK]

Convierte un modelo SQLAlchemy (o lista de modelos) a dict JSON-safe.

`require_fields`

• Parámetros: `data, fields`

• Async: —

• Complejidad: [LOW] 5

• Documentada: [OK]

> **Regresa lista de campos faltantes o vacíos.**

`now`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`serialize(obj)`

Ubicación: `utils.py`

> **Convierte un modelo SQLAlchemy (o lista de modelos) a dict JSON-safe.**

`require_fields(data, fields)`

Ubicación: `utils.py`

> **Regresa lista de campos faltantes o vacíos.**

Módulo: `auth_utils.py`

`hash_password`

• Parámetros: `plain`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`check_password`

• Parámetros: `plain, hashed`

• Async: —

• Complejidad: [LOW] 3

• Documentada: [X]

`generate_token`

• Parámetros: `user_id, rol`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`decode_token`

• Parámetros: `token`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`token_required`

• Parámetros: `f`

• Async: —

• Complejidad: [LOW] 5

• Documentada: [OK]

> **Protege un endpoint: requiere header Authorization: Bearer <token>.**

`decorated`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 5

• Documentada: [X]

`token_required(f)`

Ubicación: `auth_utils.py`

> **Protege un endpoint: requiere header Authorization: Bearer <token>.**

Módulo: `routes/historial.py`

`historial_grupo`

• Parámetros: `grupo_id`

• Async: —

• Complejidad: [MED] 7

• Documentada: [OK]

> **Combina registros de actividad, metas asignadas y notificaciones enviadas**

de todos los deportistas de un grupo, ordenados del más reciente al más antiguo.

Alimenta la pantalla 'Historial de eventos' de la app web.

`historial_grupo(grupo_id)`

Ubicación: `routes/historial.py`

> **Combina registros de actividad, metas asignadas y notificaciones enviadas**

de todos los deportistas de un grupo, ordenados del más reciente al más antiguo.

Alimenta la pantalla 'Historial de eventos' de la app web.

Módulo: `routes/auth.py`

`registro`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 5

• Documentada: [X]

`login`

• Parámetros: `—`

• Async: —

• Complejidad: [LOW] 5

• Documentada: [X]

Módulo: `routes/actividad.py`

`registrar_actividad`

• Parámetros: `—`

• Async: —

• Complejidad: [MED] 9

• Documentada: [OK]

> **Endpoint que consume la app wearable para enviar cada lectura de sensores.**

No requiere token de entrenador (el propio dispositivo se identifica con su id),

pero sí valida que el dispositivo y el deportista existan.

`actividad_por_deportista`

• Parámetros: `deportista_id`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [OK]

> **Historial + último registro de un deportista (vista individual).**

`actividad_por_grupo`

• Parámetros: `grupo_id`

• Async: —

• Complejidad: [LOW] 2

• Documentada: [OK]

> **Vista grupal: resumen agregado (último registro de cada deportista del grupo).**

`registrar_actividad()`

Ubicación: `routes/actividad.py`

> **Endpoint que consume la app wearable para enviar cada lectura de sensores.**

No requiere token de entrenador (el propio dispositivo se identifica con su id),

pero sí valida que el dispositivo y el deportista existan.

`actividad_por_deportista(deportista_id)`

Ubicación: `routes/actividad.py`

> **Historial + último registro de un deportista (vista individual).**

`actividad_por_grupo(grupo_id)`

Ubicación: `routes/actividad.py`

> **Vista grupal: resumen agregado (último registro de cada deportista del grupo).**

Módulo: `routes/notificaciones.py`

`enviar_notificacion`

• Parámetros: –

• Async: —

• Complejidad: [MED] 9

• Documentada: [OK]

> **Envía una notificación a uno o varios deportistas desde la app web.**

`notificaciones_por_deportista`

• Parámetros: `deportista_id`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [OK]

> **Consumido por la app wearable para mostrar sus notificaciones.**

No requiere token de entrenador: el propio dispositivo consulta con el id del deportista que tiene configurado.

`marcar_leida`

• Parámetros: `notif_id`

• Async: —

• Complejidad: [LOW] 2

• Documentada: [X]

`enviar_notificacion()`

Ubicación: `routes/notificaciones.py`

> **Envía una notificación a uno o varios deportistas desde la app web.**

`notificaciones_por_deportista(deportista_id)`

Ubicación: `routes/notificaciones.py`

Consumido por la app wearable para mostrar sus notificaciones.

No requiere token de entrenador: el propio dispositivo consulta con el id del deportista que tiene configurado.

Módulo: `routes/grupos.py`

`listar_grupos`

- Parámetros: –
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`crear_grupo`

- Parámetros: –
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

`actualizar_grupo`

- Parámetros: `grupo_id`
- Async: —
- Complejidad: [LOW] 4
- Documentada: [X]

`eliminar_grupo`

- Parámetros: `grupo_id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

Módulo: `routes/metas.py`

`crear_meta`

• Parámetros: `–`

• Async: —

• Complejidad: [MED] 7

• Documentada: [X]

`metas_por_deportista`

• Parámetros: `deportista_id`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`actualizar_estado_meta`

• Parámetros: `meta_id`

• Async: —

• Complejidad: [LOW] 4

• Documentada: [X]

Módulo: `routes/dispositivos.py`

`listar_dispositivos`

• Parámetros: `–`

• Async: —

• Complejidad: [LOW] 4

• Documentada: [X]

`registrar_dispositivo`

• Parámetros: `–`

• Async: —

• Complejidad: [MED] 6

• Documentada: [X]

`actualizar_estado`

• Parámetros: `dispositivo_id`

• Async: —

• Complejidad: [LOW] 4

• Documentada: [OK]

Usado por el wearable para marcarse como conectado/desconectado.

`eliminar_dispositivo`

• Parámetros: `dispositivo_id`

• Async: —

• Complejidad: [LOW] 2

• Documentada: [X]

`actualizar_estado(dispositivo_id)`

Ubicación: `routes/dispositivos.py`

Usado por el wearable para marcarse como conectado/desconectado.

Módulo: `routes/deportistas.py`

`listar_deportistas`

• Parámetros: `–`

• Async: —

• Complejidad: [LOW] 4

• Documentada: [OK]

Lista deportistas; opcionalmente filtra por ?grupo=<id_grupo>.

`obtener_deportista`

• Parámetros: `deportista_id`

• Async: —

• Complejidad: [LOW] 2

• Documentada: [X]

`crear_deportista`

• Parámetros: `–`

• Async: —

• Complejidad: [MED] 8

• Documentada: [X]

`actualizar_deportista`

• Parámetros: `deportista_id`

• Async: —

• Complejidad: [MED] 9

• Documentada: [X]

`eliminar_deportista`

• Parámetros: `deportista_id`

• Async: —

• Complejidad: [LOW] 2

• Documentada: [X]

`listar_deportistas()`

Ubicación: `routes/deportistas.py`

Lista deportistas; opcionalmente filtra por ?grupo=<id_grupo>.

Módulo: `web/js/app.js`

`async api`

• Parámetros: `path, { method = "GET", body, auth = true } = {}`

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`showToast`

• Parámetros: `message, isError = false`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`fmtFecha`

• Parámetros: `iso`

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`setupLoginListeners`

• Parámetros: **`–`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async enterApp`

• Parámetros: **`–`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`setupNavigationListeners`

• Parámetros: **`–`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`switchView`

• Parámetros: **`name`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async cargarGrupos`

• Parámetros: **`–`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`llenarSelectsDeGrupo`

• Parámetros: **`–`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`setupDashboardListeners`

• Parámetros: **`–`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async cargarDashboard`

• Parámetros: **`–`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`renderListaGrupos`

• Parámetros: **`–`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`setupDeportistasListeners`

• Parámetros: **`–`**

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async cargarPantallaDeportistas`

• Parámetros: **`–`**

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`setupActividadListeners`

• Parámetros: –

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

async cargarPantallaActividad

• Parámetros: –

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

async onActGrupoChange

• Parámetros: –

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

async cargarActividadDeportista

• Parámetros: –

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

setupNotificacionesListeners

• Parámetros: –

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

async cargarPantallaNotificaciones

• Parámetros: –

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

`setupHistorialListeners`

• Parámetros: –

• Async: —

• Complejidad: [LOW] 1

• Documentada: [X]

`async cargarPantallaHistorial`

• Parámetros: –

• Async: [ASYNC]

• Complejidad: [LOW] 1

• Documentada: [X]

---

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

• **App**

---

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

• **Lenguaje principal**: Python

• **Archivos detectados**: 20

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
```

```
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python app.py"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

• [ ] Variables de entorno configuradas

• [ ] Base de datos migrada

• [ ] Tests ejecutados y pasando

• [ ] Logs configurados

• [ ] HTTPS habilitado en producción

• [ ] Backups programados

• [ ] Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-10-13

• Versión inicial analizada por AutoDocs AI

• Documentación técnica generada automáticamente

• Análisis de calidad del código completado

---

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 75/100 (Bueno [YELLOW])

Distribución de Documentación:

• Funciones documentadas: 12/63 (19%)

• Clases documentadas: 0/8

Análisis de Complejidad:

• Complejidad promedio: 2.57

• Complejidad máxima: 9

• Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

```
[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: → Estable
```

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

• Agregar docstrings a las 51 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados. [OK]

---

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

• **Funciones sin documentar:** 51

- `create_app` en `app.py`

- `favicon` en `app.py`

- `health` en `app.py`

- `not_found` en `app.py`

- `method_not_allowed` en `app.py`

- ... y 46 más

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	8	Requieren refactorización urgente
■ Media	4–5	10	Candidatas a simplificación
■ Baja	≤ 3	45	Complejidad aceptable

Funciones de alta complejidad — detalle y recomendaciones:

• **`registrar_actividad`**

- ■ Archivo: **`routes/actividad.py`** — línea 12

- ■ Complejidad ciclomática: **9**

- ■ Sugerencia: Complejidad moderada-alta (9). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

• **`enviar_notificacion`**

- ■ Archivo: **`routes/notificaciones.py`** — línea 13

- ■ Complejidad ciclomática: **9**

- ■ Sugerencia: Complejidad moderada-alta (9). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

• **`actualizar_deportista [sin docstring]`**

- ■ Archivo: **`routes/deportistas.py`** — línea 71

- ■ Complejidad ciclomática: **9**

- ■ Sugerencia: Complejidad moderada-alta (9). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

• **`crear_deportista [sin docstring]`**

- ■ Archivo: **`routes/deportistas.py`** — línea 37

- ■ Complejidad ciclomática: **8**

- ■ Sugerencia: Complejidad moderada-alta (8). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

• **`historial_grupo`**

- ■ Archivo: **`routes/historial.py`** — línea 11

- ■ Complejidad ciclomática: **7**

- ■ Sugerencia: Complejidad moderada-alta (7). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

• **`crear_meta [sin docstring]`**

- ■ Archivo: **`routes/metas.py`** — línea 16

- ■ Complejidad ciclomática: **7**

- ■ Sugerencia: Complejidad moderada-alta (7). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

• **`serialize`**

- ■ Archivo: **`utils.py`** — línea 4

- ■ Complejidad ciclomática: **6**

- ■ Sugerencia: Complejidad moderada-alta (6). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

• **`registrar_dispositivo [sin docstring]`**

- ■ Archivo: **`routes/dispositivos.py`** — línea 27

- ■ Complejidad ciclomática: **6**

- ■ Sugerencia: Complejidad moderada-alta (6). Revisa si alguno de los bloques if/for puede extraerse a una función con nombre descriptivo.

B) Funciones Sin Documentación

51 de 63 funciones carecen de docstring (81%).

Función	Archivo	Línea	Complejidad
`create_app`	`app.py`	10	2
`favicon`	`app.py`	38	1
`health`	`app.py`	42	1
`not_found`	`app.py`	46	1
`method_not_allowed`	`app.py`	50	1
`server_error`	`app.py`	54	1
`now`	`utils.py`	30	1
`hash_password`	`auth_utils.py`	11	1
`check_password`	`auth_utils.py`	15	3
`generate_token`	`auth_utils.py`	22	1

`decode_token`	`auth_utils.py`	32	1
`decorated`	`auth_utils.py`	39	5
`registro`	`routes/auth.py`	12	5
`login`	`routes/auth.py`	42	5
`marcar_leida`	`routes/notificaciones.py`	62	2
`listar_grupos`	`routes/grupos.py`	13	1
`crear_grupo`	`routes/grupos.py`	20	3
`actualizar_grupo`	`routes/grupos.py`	34	4
`eliminar_grupo`	`routes/grupos.py`	51	2
`crear_meta`	`routes/metas.py`	16	7

... y 31 funciones más sin documentar.

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

Se detectaron **1 grupos de funciones con nombres similares** repartidas en distintos archivos:

Patrón: _por_deportista / por_deportista_ — aparece en 3 archivos distintos

Función	Archivo	Línea	Complejidad
`actividad_por_deportista`	`routes/actividad.py`	58	1
`notificaciones_por_deportista`	`routes/notificaciones.py`	46	1
`metas_por_deportista`	`routes/metas.py`	60	1

■ **Sugerencia:** Las 3 funciones `actividad_por_deportista`, `notificaciones_por_deportista`, `metas_por_deportista` comparten el patrón `por_deportista` en distintos módulos. Esto sugiere lógica duplicada. Extrae la lógica común a un servicio o función base y haz que cada módulo la llame con sus parámetros propios.

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`web/js/app.js`	22	1.0	0%	■ Sobrecargado
`app.py`	7	1.1	14%	■ OK
`auth_utils.py`	6	2.7	17%	■ OK
`routes/deportistas.py`	5	5.0	20%	■ OK
`routes/grupos.py`	4	2.5	0%	■ OK
`routes/dispositivos.py`	4	4.0	25%	■ OK
`utils.py`	3	4.0	67%	■ OK
`routes/actividad.py`	3	4.0	100%	■ OK
`routes/notificaciones.py`	3	4.0	67%	■ OK
`routes/metas.py`	3	4.0	0%	■ OK
`routes/auth.py`	2	5.0	0%	■ OK
`routes/historial.py`	1	7.0	100%	■ Revisar

■ `web/js/app.js` tiene 22 funciones. Considera dividirlo en submódulos más específicos siguiendo el principio de responsabilidad única (SRP).

E) Patrones Frecuentes Adicionales (Análisis IA)

• **create**_: 1 ocurrencias

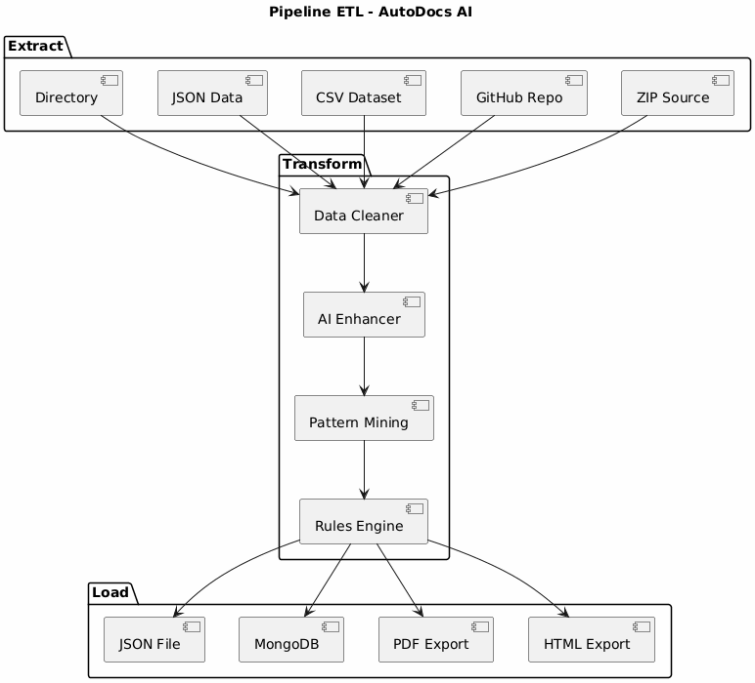
- Ejemplos: Operaciones create_ repetidas

• **check**_: 1 ocurrencias

- Ejemplos: Operaciones check_ repetidas

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):



Fases del Pipeline:

• **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales

• **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones

• **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-10-13 05:10 UTC